Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

W4-01 Der Kistenkran · Tracetabelle

a) siehe Tabelle. Zeile 5 ist eine Mischaufgabe: Das oberste Element von s (7) wandert ans Ende von q. top in Zeile 8 liest nur, pop entnimmt. Zeile 10 „rotiert“ die Schlange: Das vorderste Element wird hinten wieder angestellt. Ausgabe: 8 2

b) dequeue liefert den Typ Object; mit Object kann man nicht rechnen. Der Cast (Integer) macht daraus eine Ganzzahl (automatisches Unboxing), dann wird 1 addiert. Zeile 9 entnimmt also den vordersten Wert von q (7) und legt 7 + 1 = 8 auf s.

c) Nach Zeile 6 (s.pop()) ist s leer. Zeile 8 ruft s.top() auf einem leeren Stapel auf → Laufzeitfehler (IllegalStateException). Abfrage: if (!s.isEmpty()) { q.enqueue(s.top()); } – im Abitur müssen solche Fehler immer durch eine explizite Abfrage ausgeschlossen werden („Leer?“).

Zeile	Stapel s (oben zuerst)	Schlange q (vorn zuerst)	Ausgabe
3	4	–	
4	7, 4	–	
5	4	7	
6	2, 4	7	
7	2, 4	7, 9	
8	2, 4	7, 9, 2	
9	8, 2, 4	9, 2	
10	8, 2, 4	2, 9	
11	8, 2, 4	2, 9	8 2

W4-02 Casten am Kai · Fehler im Java-Code

a) Zeile 2: Fehler – top() liefert Object, ein Object darf nicht ohne Cast einer Variablen vom Typ Kiste zugewiesen werden (incompatible types). Zeile 3: in Ordnung, k hat den Typ Kiste. Zeile 4: Fehler – der Cast bindet schwächer als der Methodenaufruf: Zuerst wird q.head().getGewicht() ausgewertet, aber Object hat keine Methode getGewicht (cannot find symbol). Richtig: ((Kiste) q.head()).getGewicht(). Zeile 5: in Ordnung.

b) siehe Code: vor top und head jeweils die Leer-Abfrage, Rückgaben casten. Probe: s mit A(3) oben, q mit B(5) vorn → 8; beide leer → 0.

c) push erwartet Object – jede Zeichenkette ist auch ein Object, der Compiler kann den Inhaltstyp nicht prüfen. Erst zur Laufzeit schlägt der Cast (Kiste) fehl: ClassCastException (String kann nicht in Kiste umgewandelt werden). Deshalb dokumentiert man den Inhaltstyp in der Signatur („Stapel vom Inhaltstyp Kiste“) und legt nur passende Objekte hinein.

```
public static int obenUndVorn(Stack s, Queue q) {
    int summe = 0;
    if (!s.isEmpty()) {
        Kiste k = (Kiste) s.top();
        summe = summe + k.getGewicht();
    }
    if (!q.isEmpty()) {
        summe = summe + ((Kiste) q.head()).getGewicht();
    }
    return summe;
}
```

W4-03 Die Ladeliste wächst · Tracetabelle

a) siehe Tabelle. insertAt schiebt das Element an der Position und alle folgenden nach hinten, setItem ersetzt nur, delete schiebt die folgenden nach vorn. insertAt(d.getLength(), …) hängt am Ende an. Ausgabe von Zeile 9: 4 Boje

b) A: Laufzeitfehler – die gültigen Indizes sind 0 bis 3, delete(4) greift auf eine nicht existierende Position zu (IndexOutOfBoundsException). B: kein Fehler – 4 ist die Länge, insertAt hängt dann am Ende an (Fass, Boje, Netz, Kran, Mast). C: kein Fehler – getItem(3) liefert "Kran"; der Cast (String) ist nötig, weil getItem Object liefert.

c) vor getItem, setItem und delete: if (i >= 0 && i < d.getLength()) – vor insertAt: if (i >= 0 && i <= d.getLength()). Typischer Abi-Fehler: <= bei getItem (ein Element zu weit).

```
Zeile | d (Index 0 zuerst)
------+--------------------------
  2   | Anker
  3   | Anker, Seil
  4   | Fass, Anker, Seil
  5   | Fass, Anker, Netz, Seil
  6   | Fass, Boje, Netz, Seil
  7   | Fass, Boje, Netz
  8   | Fass, Boje, Netz, Kran
Ausgabe Zeile 9: 4 Boje
```

W4-04 Das Zugbuch · Funktionsweise erläutern

a) siehe Tabelle. Rückgängig gemacht wird immer der zuletzt ausgeführte Zug, wiederholt immer der zuletzt rückgängig gemachte.

b) Nach einem neuen Zug gehören die zuvor rückgängig gemachten Züge zu einer „anderen Geschichte“ – sie dürfen nicht mehr wiederholt werden. Ohne das Leeren läge nach zug("W") noch S auf zurueckgenommen; wiederholen() würde "S" liefern und S auf W legen – ein Zug, der in diesem Spielverlauf gar nicht vorkommt.

c) Rückgängig gemacht wird in umgekehrter Reihenfolge der Ausführung: Der zuletzt ausgeführte Zug zuerst (LIFO). Genau das leistet ein Stapel; eine Schlange (FIFO) würde den ältesten Zug liefern. Auch Redo ist LIFO: Zuletzt rückgängig gemacht → zuerst wiederholt.

```
Aufruf          | Rückgabe | erledigt (oben zuerst) | zurueckgenommen (oben
zuerst)
----------------+----------+------------------------
+-----------------------------
zug("N")        |          | N                      | –
zug("O")        |          | O, N                   | –
zug("S")        |          | S, O, N                | –
rueckgaengig()  | "S"      | O, N                   | S
rueckgaengig()  | "O"      | N                      | O, S
wiederholen()   | "O"      | O, N                   | S
zug("W")        |          | W, O, N                | –
wiederholen()   | ""       | W, O, N                | –
rueckgaengig()  | "W"      | O, N                   | W
```

W4-05 Die richtige Struktur für den Hafen · Methodenkopf und Signatur

a) (1) Schlange: Wer zuerst ankommt, wird zuerst geschleust (FIFO); angehängt wird hinten, entnommen vorn. (2) Stapel: Die zuletzt eingeladene Kiste steht an der Tür und kommt zuerst heraus (LIFO). (3) Dynamische Reihung: Einfügen und Entfernen an beliebiger Position und direkter Zugriff über den Index – mit Stapel oder Schlange müsste man dafür alle Elemente umschichten.

b) siehe Code. Der Rückgabetyp „dynamische Reihung vom Inhaltstyp Kiste“ wird in Java einfach DynArray.

c) passt(liste: dynamische Reihung vom Inhaltstyp …, wartend: Schlange vom Inhaltstyp …, faktor: Kommazahl): Wahrheitswert. Der Java-Kopf verrät den Inhaltstyp nicht (die Klassen speichern Object). Welche Klasse man beim Casten angeben muss, steht nur in der Signatur bzw. in der Aufgabenstellung – deshalb dort immer „vom Inhaltstyp …“ angeben.

```
public static Schiff naechstesSchiff(Queue schleuse)

public static DynArray ladeliste(Stack container, int grenze)
```

W4-06 Inventur ohne Chaos · Lückencode

a) (1) new Stack() · (2) !s.isEmpty() · (3) (Kiste) s.pop() · (4) hilf.push(k) · (5) s.push(hilf.pop()). Muster: alles auf einen Hilfsstapel umschichten (dabei auswerten), danach zurückschichten – zweimal umgedreht ist wieder die alte Reihenfolge.

b) Rückgabe 2 (B und C), aber s = C, B, A (oben zuerst): Die Reihenfolge ist umgekehrt. Die Schlange gibt die Kisten in der Reihenfolge zurück, in der sie entnommen wurden (A zuerst) – A wird also zuerst gepusht und landet ganz unten. Gezählt wird richtig, aber „unverändert“ ist verletzt.

c) top() liefert nur die oberste Kiste. An die darunterliegenden kommt man nur, indem man die oberen entnimmt – deshalb braucht man eine zweite Struktur zum Zwischenlagern.

```
public static int anzahlSchwerer(Stack s, int grenze) {
    Stack hilf = new Stack();
    int anzahl = 0;
    while (!s.isEmpty()) {
        Kiste k = (Kiste) s.pop();
        if (k.getGewicht() > grenze) {
            anzahl++;
        }
        hilf.push(k);
    }
    while (!hilf.isEmpty()) {
        s.push(hilf.pop());
    }
    return anzahl;
}
```

W4-07 Der Wechselzauber · Struktogramm lesen

a) Nach der ersten Schleife: s = E, D, C, B, A (oben zuerst), q leer, n = 5. Nach der zweiten Schleife: q = E, D, C, B, A, s leer – Schlange → Stapel → Schlange kehrt die Reihenfolge um. Die Zählschleife läuft 5 / 2 = 2-mal und stellt die beiden vordersten Elemente hinten an: q = C, B, A, E, D.

b) A, B, C, D → umgekehrt D, C, B, A → 2-mal rotiert: B, A, D, C. Allgemein: q wird in einen vorderen Teil mit n − n / 2 und einen hinteren Teil mit n / 2 Elementen geteilt; jeder Teil wird für sich umgekehrt, die Teile bleiben an ihrem Platz (A, B, C | D, E → C, B, A | E, D).

c) Bei leerer Schlange ist die Bedingung der ersten Schleife sofort falsch, s bleibt leer, die zweite Schleife läuft nicht, n = 0 – die Zählschleife läuft für i von 1 bis 0 gar nicht. dequeue und pop werden nur aufgerufen, nachdem isEmpty geprüft bzw. n Elemente gezählt wurden.

	Stapel s (oben zuerst)	Schlange q (vorn zuerst)
nach der 1. Schleife:	E, D, C, B, A	–
nach der 2. Schleife:	–	E, D, C, B, A
am Ende (2 x rotiert):	–	C, B, A, E, D

W4-08 Die Lotsenliste · Methode implementieren

a) siehe Code (Muster „Hilfsschlange“, vgl. Abi 2023: pruefeRing). Probe: 2, 4, 4, 9 → wahr; 4, 2, 7 → falsch; leer → wahr; q danach jeweils unverändert. Gleichwertig: alle Werte dequeue und sofort wieder enqueue, wenn die Länge vorher gezählt wurde (Rotieren), oder ein erster Wert mit head() vor der Schleife – dann aber nur nach Leer-Abfrage.

b) Rückgabe falsch – aber q = 7: Die 4 liegt nur in hilf (die mit der Methode verschwindet), die 2 wurde entnommen und nirgends abgelegt. Ein vorzeitiges return verlässt die Methode, bevor die Schlange wiederhergestellt ist. Deshalb: Ergebnis in einer Variablen merken und erst nach dem Zurückschichten zurückgeben.

c) dequeue nur in der Schleife mit der Bedingung !q.isEmpty() bzw. !hilf.isEmpty(); der Vergleich mit dem Vorgänger erst ab dem zweiten Wert (Variable erstes) – deshalb ist auch kein head() auf einer leeren Schlange nötig.

```
public static boolean istAufsteigend(Queue q) {
    Queue hilf = new Queue();
    boolean aufsteigend = true;
    boolean erstes = true;
    int vorher = 0;
    while (!q.isEmpty()) {
        int x = (Integer) q.dequeue();
        if (!erstes && x < vorher) {
            aufsteigend = false;
        }
        erstes = false;
        vorher = x;
        hilf.enqueue(x);
    }
    while (!hilf.isEmpty()) {
        q.enqueue(hilf.dequeue());
    }
    return aufsteigend;
}
```

W4-09 Der Ladekran verrutscht · Fehler im Java-Code

a) siehe Tabelle. Bei i = 4 ist die Länge nur noch 4; wegen <= wird getItem(4) aufgerufen → Laufzeitfehler (IndexOutOfBoundsException). Gültige Indizes sind 0 bis getLength() − 1.

b) d = B(2), C(8), E(1), F(9): B und E sind zu leicht, bleiben aber. Nach delete(i) rückt der Nachfolger auf den Index i nach – die Schleife erhöht i trotzdem und überspringt ihn. Der Fehler wirkt sich immer aus, wenn zwei zu leichte Kisten direkt aufeinander folgen.

c) (1) Rückwärts löschen: Beim Löschen verschieben sich nur Elemente mit größerem Index – die sind schon geprüft. (2) Vorwärts mit while: i nur im sonst-Fall erhöhen. Beide liefern C(8), F(9).

```
i | Länge | getItem(i) | Aktion      | d danach
--+-------+------------+-------------+-------------------------
0 |   6   | A(3)       | delete(0)   | B, C, D, E, F
1 |   5   | C(8)       | –           | B, C, D, E, F
2 |   5   | D(1)       | delete(2)   | B, C, E, F
3 |   4   | F(9)       | –           | B, C, E, F
4 |   4   | Fehler!    | Abbruch     | (IndexOutOfBoundsException)
```

```
// (1) rueckwaerts
for (int i = d.getLength() - 1; i >= 0; i--) {
    if (((Kiste) d.getItem(i)).getGewicht() < grenze) {
        d.delete(i);
    }
}

// (2) vorwaerts, i nur erhoehen, wenn nicht geloescht
int i = 0;
while (i < d.getLength()) {
    if (((Kiste) d.getItem(i)).getGewicht() < grenze) {
        d.delete(i);
    } else {
        i++;
    }
}
```

W4-10 Sortiert verladen · Tracetabelle

a) siehe Tabelle. j zählt, wie viele Kisten höchstens so schwer sind wie k – genau dort wird k eingefügt.

b) Beim Einfügen von R(5) werden Q(2) und P(5) übersprungen, weil 2 ≤ 5 und 5 ≤ 5 gilt → j = 2, R steht hinter P. Gleich schwere Kisten behalten so ihre Ankunftsreihenfolge (stabil). Mit < bricht die Schleife bei P ab (5 < 5 ist falsch) → j = 1, R steht vor P.

c) && wertet von links nach rechts aus und bricht ab, sobald der linke Teil falsch ist (Kurzschluss). Nur so wird getItem(j) nie mit j = getLength() aufgerufen. Vertauscht stürzt jeder Aufruf ab, bei dem k ans Ende gehört – schon der erste Aufruf mit leerem d (getItem(0)) und z. B. das Einfügen von S(9) → IndexOutOfBoundsException. insertAt(j, k) mit j = getLength() ist dagegen erlaubt (hängt an).

```
Aufruf | j | d danach (Index 0 zuerst)
-------+---+--------------------------
P(5)   | 0 | P(5)
Q(2)   | 0 | Q(2), P(5)
R(5)   | 2 | Q(2), P(5), R(5)
S(9)   | 3 | Q(2), P(5), R(5), S(9)
T(1)   | 0 | T(1), Q(2), P(5), R(5), S(9)
```

W4-11 Die leichteste Kiste zuerst · Code ↔ Struktogramm

a) siehe Code. Die Zählschleife „für i von 1 bis d.getLength() − 1“ wird zu for (int i = 1; i < d.getLength(); i++). Ohne die Leer-Abfrage würde bei leerem d getItem(0) einen Laufzeitfehler auslösen.

b) Rückgabe B(2); d = A(4), C(7), D(2). Bei Gleichstand wird die erste leichteste Kiste entfernt, weil nur bei echt kleinerem Gewicht (<) ein neuer minIndex gesetzt wird.

c) delete braucht einen Index. Mit der Kiste allein müsste man ihre Position erneut suchen. Nach delete(minIndex) steht an dieser Position bereits die nächste Kiste (oder keine) – die Rückgabe muss deshalb vorher gesichert werden.

```
public static Kiste entferneLeichteste(DynArray d) {
    if (d.isEmpty()) {
        return null;
    }
    int minIndex = 0;
    for (int i = 1; i < d.getLength(); i++) {
        if (((Kiste) d.getItem(i)).getGewicht()
                < ((Kiste) d.getItem(minIndex)).getGewicht()) {
            minIndex = i;
        }
    }
    Kiste leichteste = (Kiste) d.getItem(minIndex);
    d.delete(minIndex);
    return leichteste;
}
```

W4-12 Klammern im Logbuch · Tracetabelle

a) siehe Tabelle. Jede schließende Klammer muss zur zuletzt geöffneten passen – der Stapel liefert genau diese (LIFO). Am Ende liegt noch (auf dem Stapel → Rückgabe falsch (3).

b) "([)]": falsch (2) – bei) liegt [oben. ")(": falsch (1) – schließende Klammer bei leerem Stapel; ohne die Leer-Abfrage gäbe es hier einen Laufzeitfehler. "(()": falsch (3) – am Ende ist der Stapel nicht leer.

c) push erwartet Object; das char wird automatisch in ein Character-Objekt verpackt (Autoboxing). pop liefert Object – der Cast (Character) macht daraus wieder ein Character, das beim Zuweisen an char automatisch ausgepackt wird.

```
Zeichen | Stapel s (oben zuerst)
--------+----------------------
   {    | {
   [    | [, {
   (    | (, [, {
   )    | [, {
   ]    | {
   }    | –
   (    | (
Rückgabe: false
```

W4-13 Der Postfix-Rechner des Zollamts · Tracetabelle

a) siehe Tabelle. Ziffern werden gepusht ((int) c – (int) '0' wandelt das Zeichen über den ASCII-Wert in die Zahl um). Ein Rechenzeichen entnimmt zwei Werte – der zuerst entnommene ist der rechte Operand – und legt das Ergebnis zurück. Rückgabe: 13.

b) (5 − 2) · 3 + 4 = 13. Vertauscht: 2 − 5 = −3, dann 3 · (−3) = −9, dann 4 + (−9) = −5 – bei − (nicht kommutativ) wird das Ergebnis falsch.

c) z. B. "5+": Bei + liegt nur 5 auf dem Stapel – das zweite pop (links) wird auf dem leeren Stapel ausgeführt → Laufzeitfehler (IllegalStateException). Absichern: vor jedem pop s.isEmpty() prüfen und z. B. einen Fehlerwert zurückgeben; am Ende prüfen, ob genau ein Wert übrig ist.

```
Zeichen | Stapel s (oben zuerst)
--------+-----------------------
   5    | 5
   2    | 2, 5
   -    | 3
   3    | 3, 3
   *    | 9
   4    | 4, 9
   +    | 13
Rückgabe: 13
```

W4-14 Die Flut im Hafenbecken · Funktionsweise erläutern

a) Schlange: siehe linke Karte. Nach der dritten Entnahme ((1|0), (0|0), (1|1)) enthält die Schlange (2|0), (0|1), (2|1). Die Nummern wachsen mit dem Abstand von S: erst alle direkten Nachbarn (1–3), dann die mit Abstand 2 (4, 5), usw.

b) Stapel: siehe rechte Karte. Entnommen wird immer die zuletzt markierte Position: Die Flut läuft erst unten herum (3, 4, 5, 6), dann rechts hoch und oben zurück (7–10) – wie ein Weg in die Tiefe (ähnlich der Rekursion in Welt 3).

c) Jedes erreichbare '.' wird genau einmal markiert und in den Speicher gelegt; jede Position wird genau einmal entnommen und ihre Nachbarn werden geprüft – unabhängig von der Reihenfolge. Beide liefern 10. Nur die Reihenfolge der Entnahme unterscheidet sich: Schlange FIFO → Welle (Breitensuche), Stapel LIFO → Tiefensuche. Markieren beim Hineinlegen verhindert, dass ein Feld mehrfach in den Speicher kommt (z. B. (0|1) als Nachbar von (0|0) und von (1|1)).

```
Schlange:                Stapel:
    s:  0  1  2  3           s:  0  1  2  3
z = 0:  1  4  6  8       z = 0:  1 10  9  8
z = 1:  S  2  #  10      z = 1:  S  2  #  7
z = 2:  3  5  7  9       z = 2:  3  4  5  6
```